

گزارش سوال ۳ عملی - قسمت ۲ - تمرین ۳ - سیستم عامل

در این قسمت، ابتدا به task_struct ۲ فیلد ۶۴ بیتی برای ذخیره زمان ورود و خروج از فرایند اضافه کردیم:

```
#endif
    /* Matin's new field */
    int new_field;
    u64 syscall_enter_time;
    u64 syscall_exit_time;

    unsigned int __state;
```

سپس در فایل arch/x86/entry/common.c که اجرای فراخوانی های سیستمی از آن می گذرد، دو خط اضافه کردیم

تا قبل از شروع فراخوانی سیستمی و بلافاصله پس از اجرای آن، مقدار دو فیلد بالا را تعیین کند:

```
_visible noinstr void do_syscall_64(struct pt_regs *regs, int nr)
{
    add_random_kstack_offset();
    nr = syscall_enter_from_user_mode(regs, nr);

    // Record kernel entry time (B)
    current->syscall_enter_time = ktime_get_ns();

    instrumentation_begin();

    if (!do_syscall_x64(regs, nr) && !do_syscall_x32(regs, nr) && nr != -1) {
        /* Invalid system call, but still a system call. */
        regs->ax = __x64_sys_ni_syscall(regs);
    }

    instrumentation_end();

    // Record kernel exit time (C)
    current->syscall_exit_time = ktime_get_ns();

    syscall_exit_to_user_mode(regs);
}
#endif
```

برای خواندن مقادیر بالا و ارسال آنها به فضای کاربر، از `syscall` زیر استفاده می‌کنیم:

```
GNU nano 8.4 kernel/get_syscall_times.c
#include <linux/syscalls.h>
#include <linux/uaccess.h>
#include <linux/sched.h>

struct syscall_times {
    u64 enter_time;
    u64 exit_time;
};

SYSCALL_DEFINE1(get_syscall_times, struct syscall_times __user *, user_buf)
{
    struct syscall_times times;

    times.enter_time = current->syscall_enter_time;
    times.exit_time = current->syscall_exit_time;

    if (copy_to_user(user_buf, &times, sizeof(times)))
        return -EFAULT;

    // Reset for next measurement
    current->syscall_enter_time = 0;
    current->syscall_exit_time = 0;

    return 0;
}
```

آن را به جدول فراخوانی های سیستمی اضافه می‌کنیم:

```
450      common  set_mempolicy_home_node sys_set_mempolicy_home_node

# Martin's new syscall for OS-HW3-Q3
451      common  hello_syscall      sys_hello_syscall
452      common  set_new_field      sys_set_new_field
453      common  get_new_field      sys_get_new_field
454      common  get_syscall_times   sys_get_syscall_times[]

#
```

همچنین object آن را اضافه می کنیم:

```
GNU nano 8.4 kernel/Makefile *
# SPDX-License-Identifier: GPL-2.0
#
# Makefile for the linux kernel.
#

obj-y      = fork.o exec_domain.o panic.o \
            cpu.o exit.o softirq.o resource.o \
            sysctl.o capability.o ptrace.o user.o \
            signal.o sys.o umh.o workqueue.o pid.o task_work.o \
            extable.o params.o \
            kthread.o sys_ni.o nsproxy.o \
            notifier.o ksysfs.o cred.o reboot.o \
            async.o range.o smpboot.o ucount.o regset.o

obj-y += hello_syscall.o
obj-y += new_syscalls.o
obj-y += get_syscall_times.o

obj-$(CONFIG_USERMODE_DRIVER) += usermode_driver.o
obj-$(CONFIG_MODULES) += kmod.o
obj-$(CONFIG_MULTIUSER) += groups.o

ifdef CONFIG_FUNCTION_TRACER
```

مانند تمرین قبل، هسته جدید را build کرده و پس از ری بوت به آن هسته می رویم. حال کد سطح کاربر زیر را نوشته، کامپایل و اجرا می کنیم:

```
#include <iostream>
#include <unistd.h>
#include <sys/syscall.h>
#include <fcntl.h>
#include <cstring>
#include <stdint>    // <<<<< for uint64_t
#include <time.h>

#define SYS_GET_TIMES 454    // my getter syscall
#define SYS_CUSTOM 451    // my custom syscall

struct syscall_times {
    unsigned long long enter_time;
    unsigned long long exit_time;
};

// Helper function to get nanoseconds since boot using CLOCK_MONOTONIC
uint64_t monotonic_ns() {
```

```

    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return ts.tv_sec * 1000000000ULL + ts.tv_nsec;
}

// Helper to measure a syscall
template<typename Func>
void measure_syscall(const std::string &name, Func f) {
    syscall_times times;

    uint64_t A = monotonic_ns(); // user-space enter
    f();                          // perform syscall
    uint64_t D = monotonic_ns(); // user-space exit

    // Get kernel times
    if (syscall(SYS_GET_TIMES, x) != 0) {
        perror("SYS_GET_TIMES failed");
        return;
    }

    uint64_t B = times.enter_time;
    uint64_t C = times.exit_time;

    std::cout << "=== " << name << " ===\n";
    std::cout << "A (user enter) : " << A << " ns\n";
    std::cout << "B (kernel enter): " << B << " ns\n";
    std::cout << "C (kernel exit) : " << C << " ns\n";
    std::cout << "D (user exit)   : " << D << " ns\n";

    std::cout << "User->Kernel      : " << (B - A) << " ns\n";
    std::cout << "Kernel Time       : " << (C - B) << " ns\n";
    std::cout << "Kernel->User      : " << (D - C) << " ns\n";
    std::cout << "Total Syscall    : " << (D - A) << " ns\n\n";
}

int main() {
    char buf[1] = {0};

    // Measure read(0, buf, 1)
    measure_syscall("read", [&]() {
        ssize_t ret = read(0, buf, 1);
        if (ret < 0) perror("read failed");
    });

    // Measure write(1, buf, 1)
    measure_syscall("write", [&]() {

```

```

        ssize_t ret = write(1, buf, 1);
        if (ret < 0) perror("write failed");
    });

    // Measure getpid()
    measure_syscall("getpid", [&]() {
        pid_t pid = getpid();
        (void)pid;
    });

    // Measure custom syscall 451
    measure_syscall("custom 451", [&]() {
        if (syscall(SYS_CUSTOM) != 0) {
            // perror("custom syscall failed");
        }
    });

    return 0;
}

```

```

g++ -O2 -std=c++17 measure_syscalls.cpp -o measure_syscalls
sudo ./measure_syscalls

```

توجه داریم زمان های استفاده شده به ns و monotonic هستند، یعنی از زمان boot سیستم محاسبه شده اند. نتیجه اجرا به صورت زیر است:

```
bagheri-timarchi@debian:~/Desktop$ uname -r
6.1.156
bagheri-timarchi@debian:~/Desktop$ sudo ./measure_syscalls
I'm taking too long to write sth for the read syscall :)
=== read ===
A (user enter) : 257719760533 ns
B (kernel enter): 279075990008 ns
C (kernel exit) : 279075997249 ns
D (user exit)   : 279076138371 ns
User->Kernel   : 21356229475 ns
Kernel Time    : 7241 ns
Kernel->User   : 141122 ns
Total Syscall  : 21356377838 ns

I=== write ===
A (user enter) : 279076243547 ns
B (kernel enter): 279076246012 ns
C (kernel exit) : 279076246258 ns
D (user exit)   : 279076246441 ns
User->Kernel   : 2465 ns
Kernel Time    : 246 ns
Kernel->User   : 183 ns
Total Syscall  : 2894 ns

=== getpid ===
A (user enter) : 279076265265 ns
B (kernel enter): 279076268101 ns
C (kernel exit) : 279076268329 ns
D (user exit)   : 279076268517 ns
User->Kernel   : 2836 ns
Kernel Time    : 228 ns
Kernel->User   : 188 ns
Total Syscall  : 3252 ns

This is the second version of the syscall.
Saying hi directly to the user space via stdout!
=== custom 45l ===
A (user enter) : 279076293189 ns
B (kernel enter): 279076299388 ns
C (kernel exit) : 279076299634 ns
D (user exit)   : 279076299833 ns
User->Kernel   : 6199 ns
Kernel Time    : 246 ns
Kernel->User   : 199 ns
Total Syscall  : 6644 ns

bagheri-timarchi@debian:~/Desktop$
```

مشاهده می‌شود که زمان کل اجرای دستور `read` بسیار بیشتر از دیگر دستورات است، به این علت که دستور `read` منتظر

نوشتن ورودی توسط کاربر و سپس `enter` می‌ماند و ما این کار را در زمان نسبتاً طولانی انجام دادیم.

همچنین در تمام دستورات اکثر زمان کل مربوط به انتقال اولیه از فضای کاربر به فضای هسته است. دستوراتی که بررسی

کردیم عبارت‌اند از: `read`, `write`, `getpid` و دستور اختصاصی خودمان برای پرینت از فضای هسته بر روی فضای کاربر

برای امکان مقایسه بهتر، در جدول زیر دستور `read` را که زمان بسیار بیشتری داشت در نظر نگرفتیم.

